DOCUMENTATION PROJET SQL

Documentation visuelle (screenshots) pour tout le projet SQL, étape par étape pour démontrer la compréhension et la bonne éxecution du sujet.

Sommaire :

Installation de la VM

I. Le serveur

II. Les tables et les données

III. Les droits

IV. Les vues

V. Les Index

VI. Les triggers

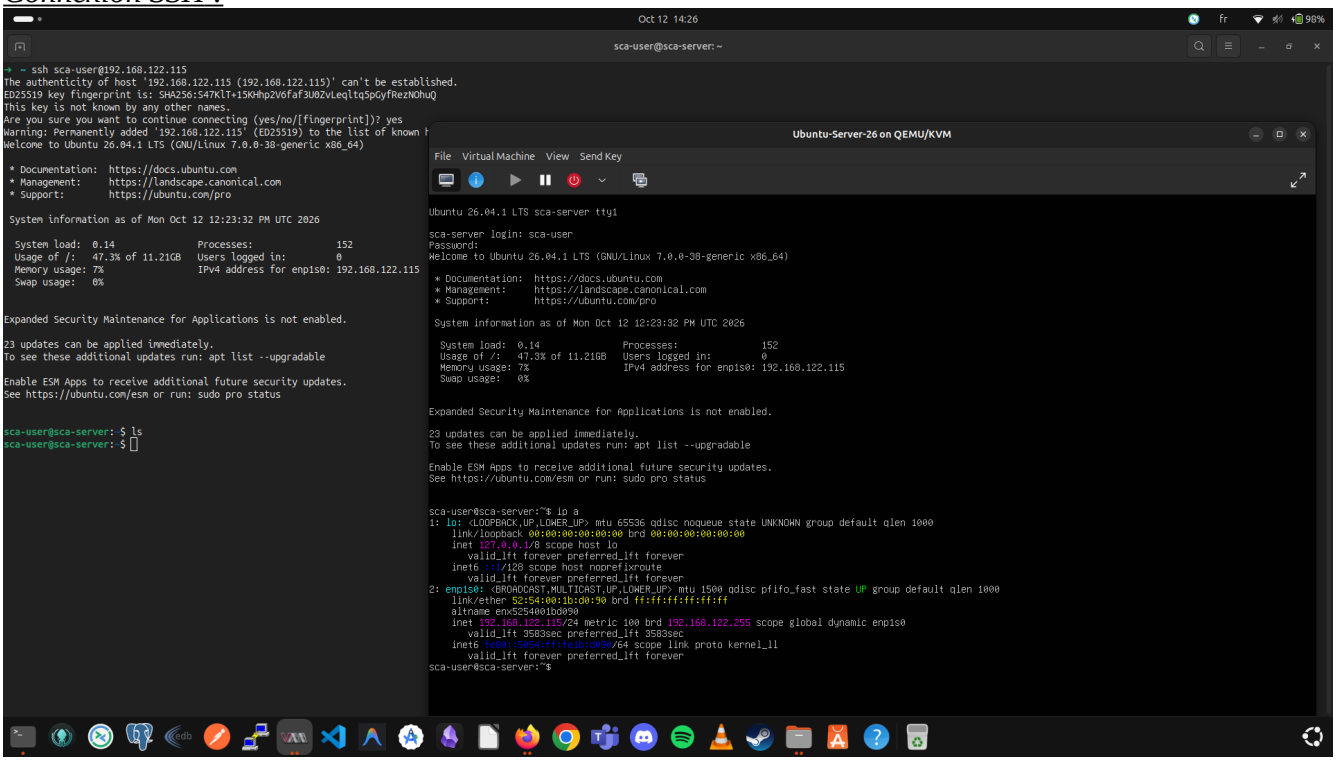
VII. Les procédures

VIII. Tests

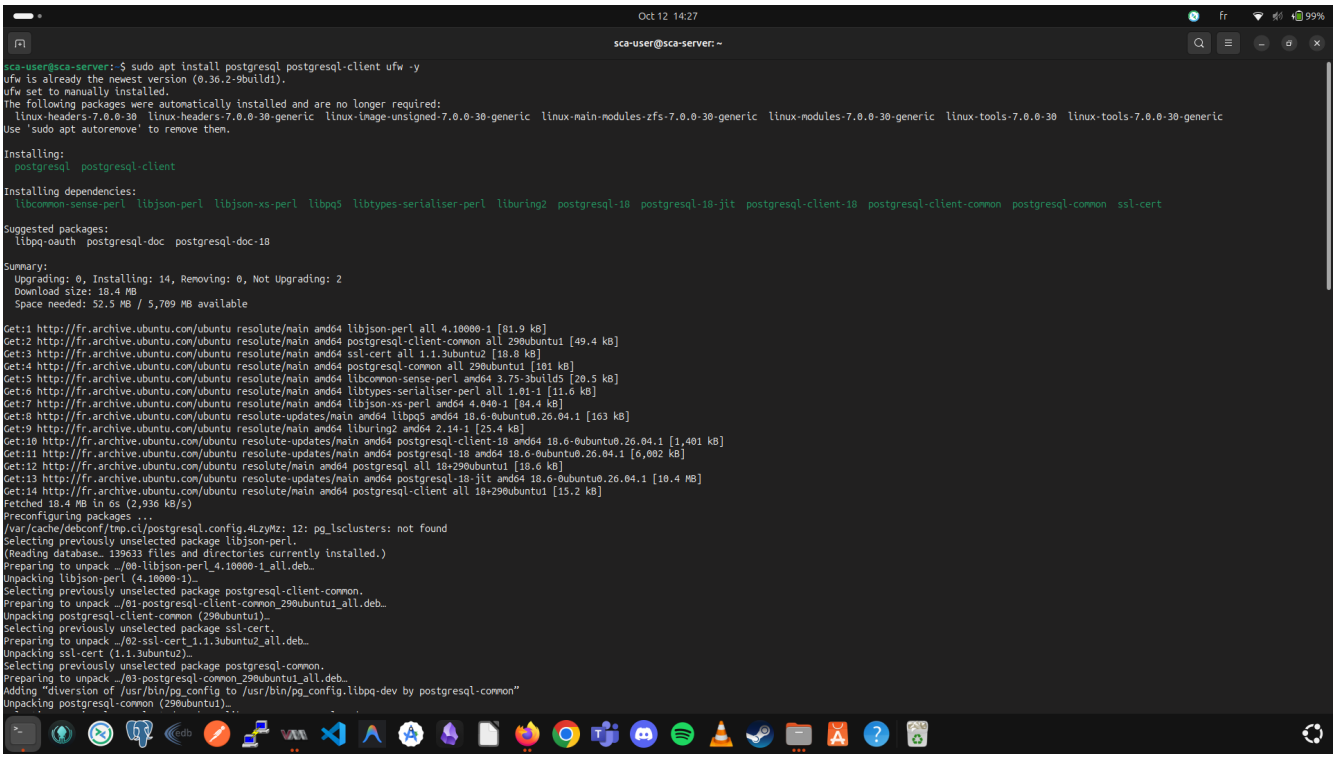
BONUS

Installation de la VM

Connexion SSH :

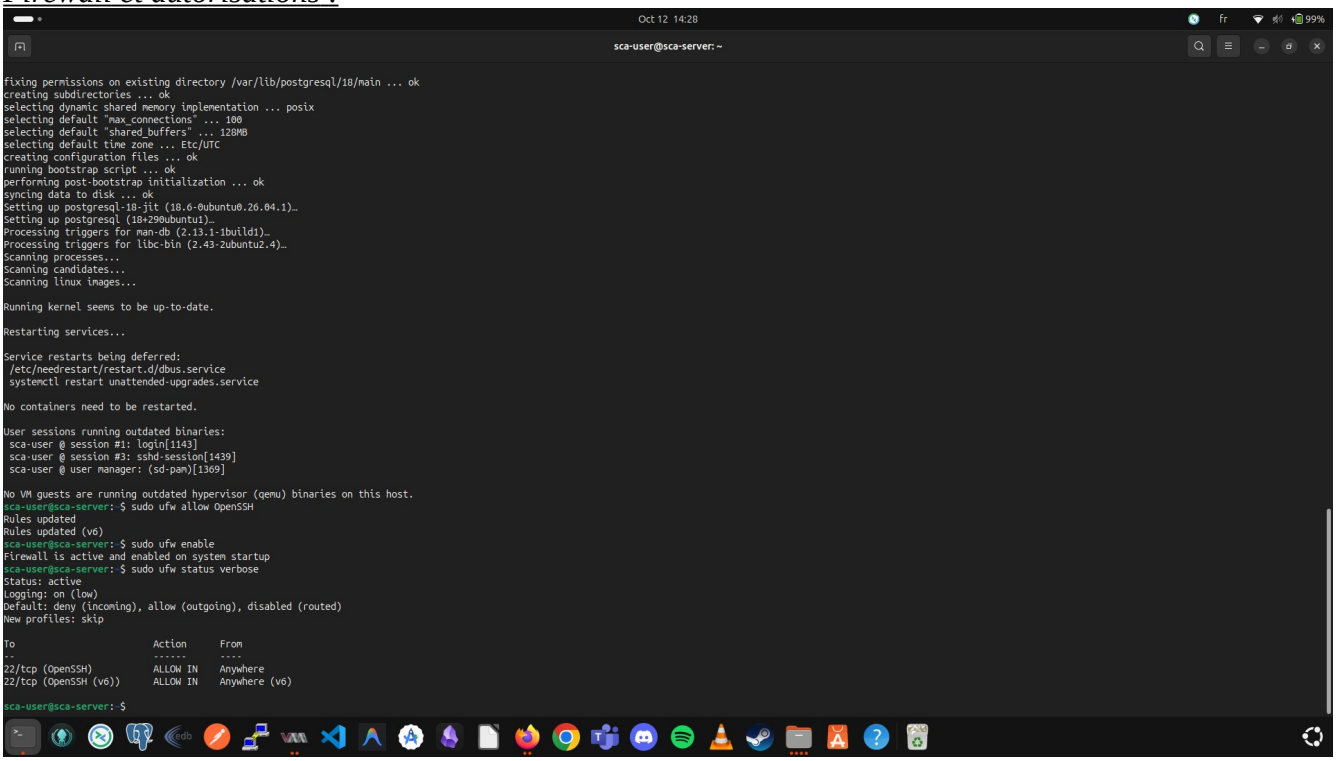


Installation POSTGRESQL :

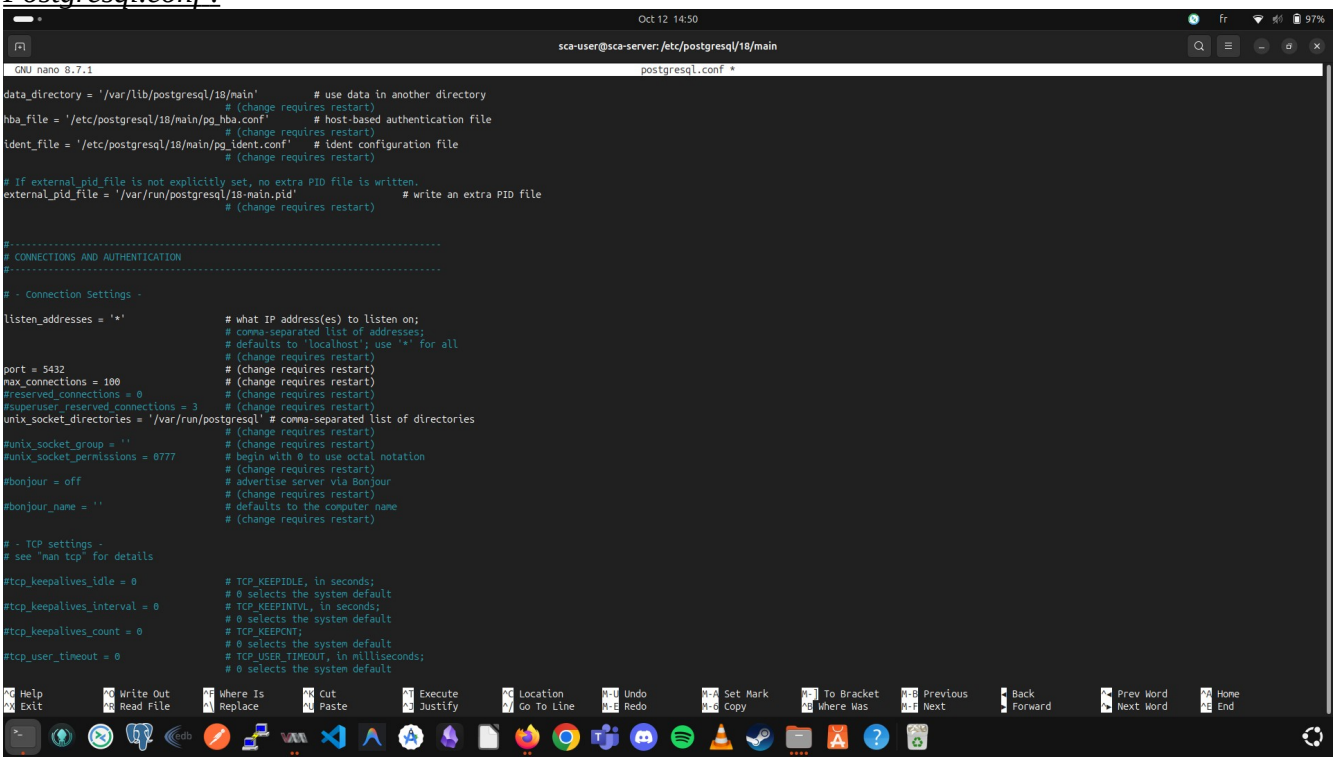


I. Le serveur

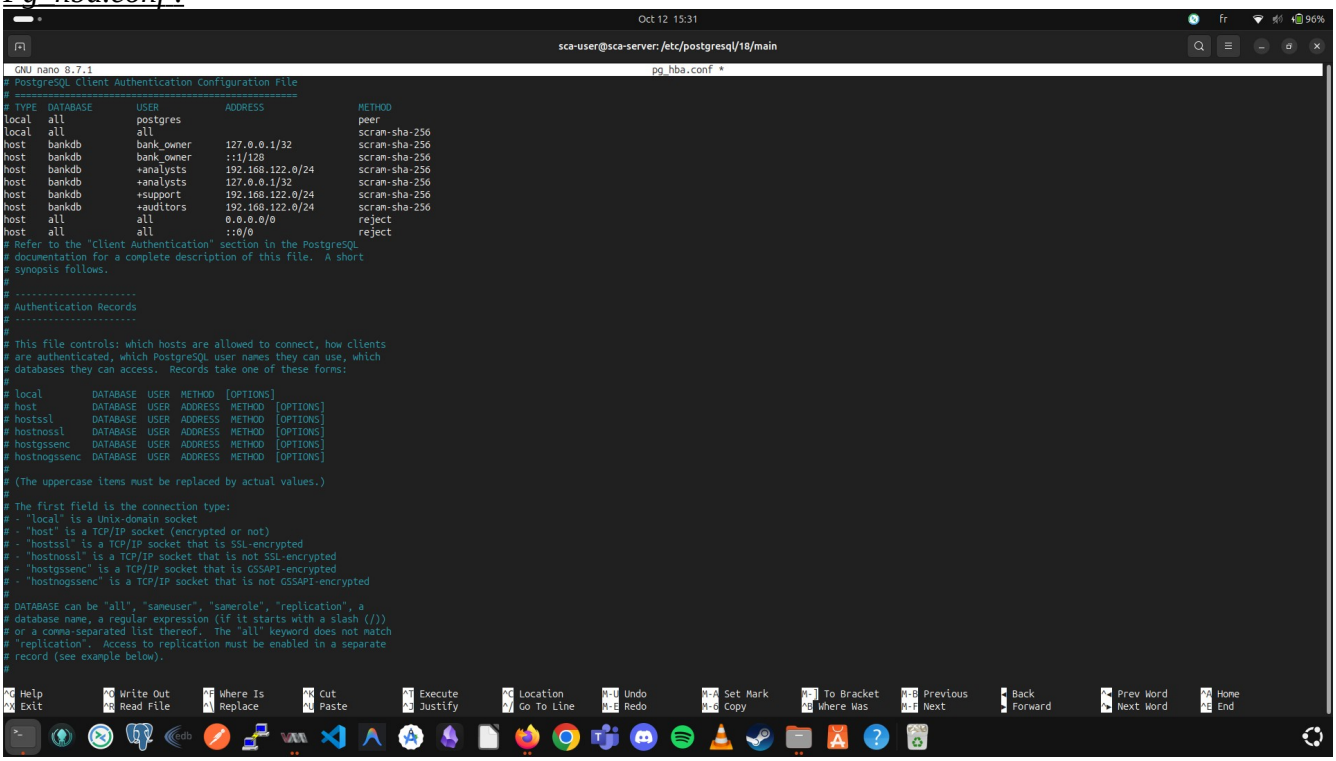
Firewall et autorisations :



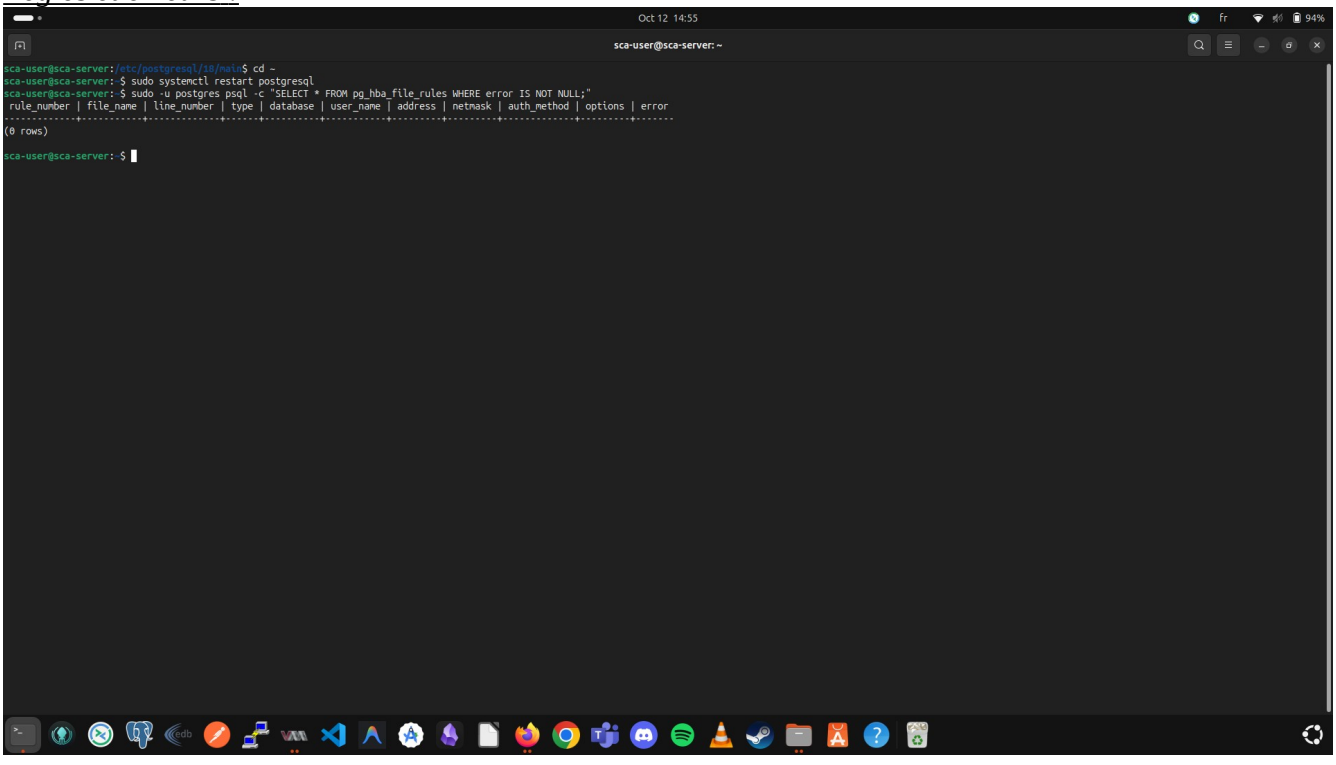
Postgresql.conf :



Pg_hba.conf :

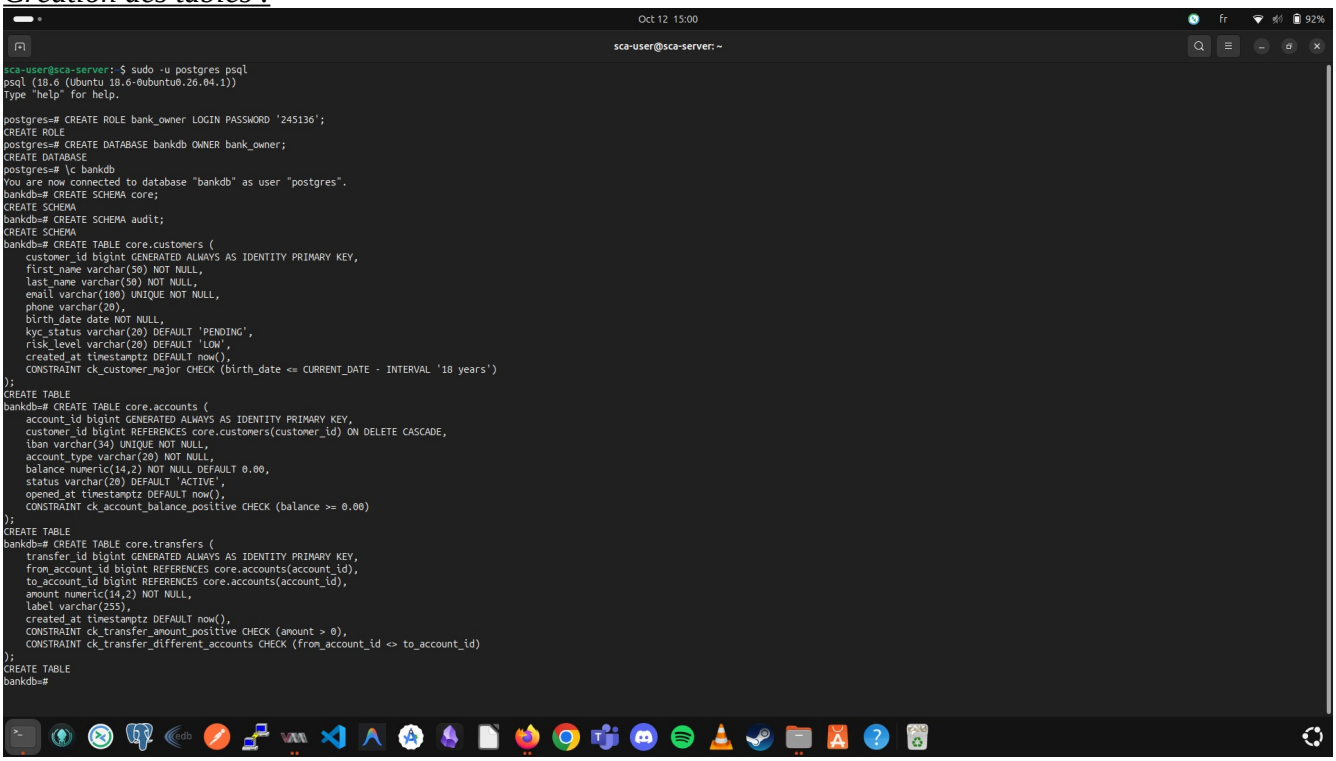


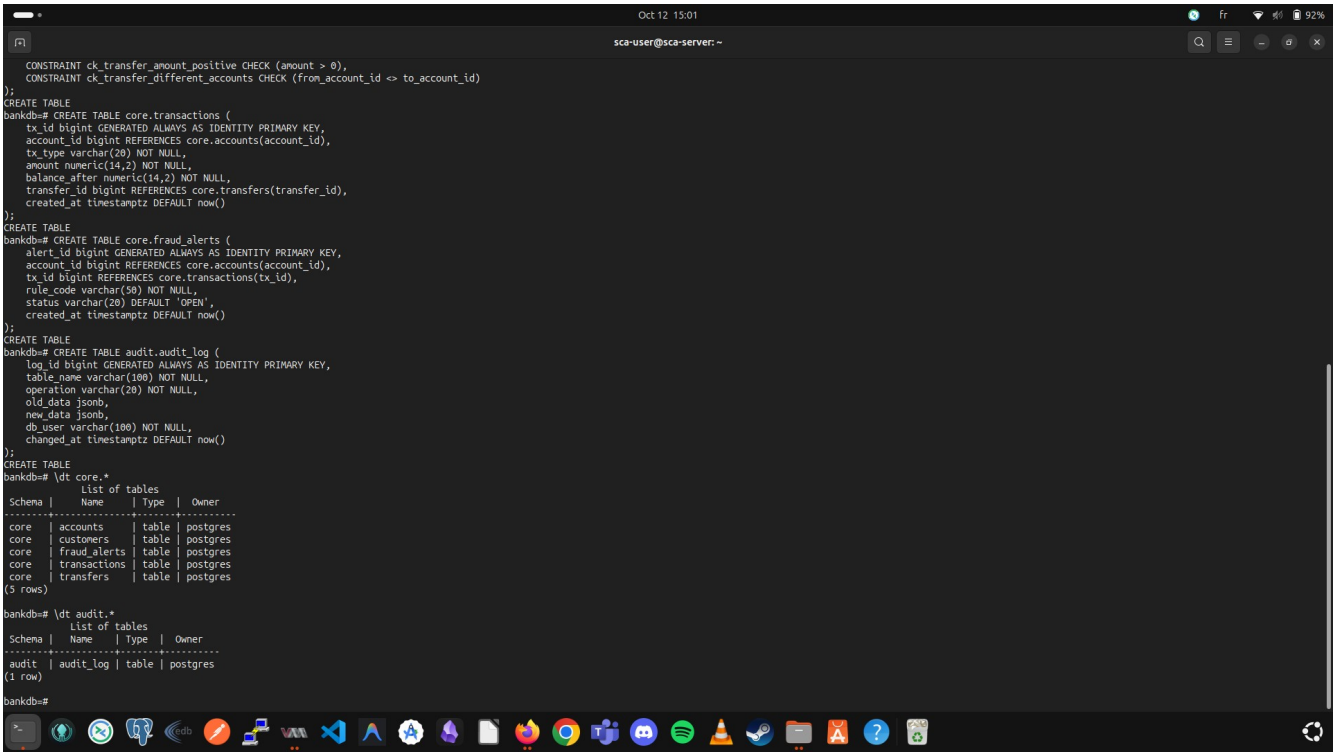
Règles et erreurs :



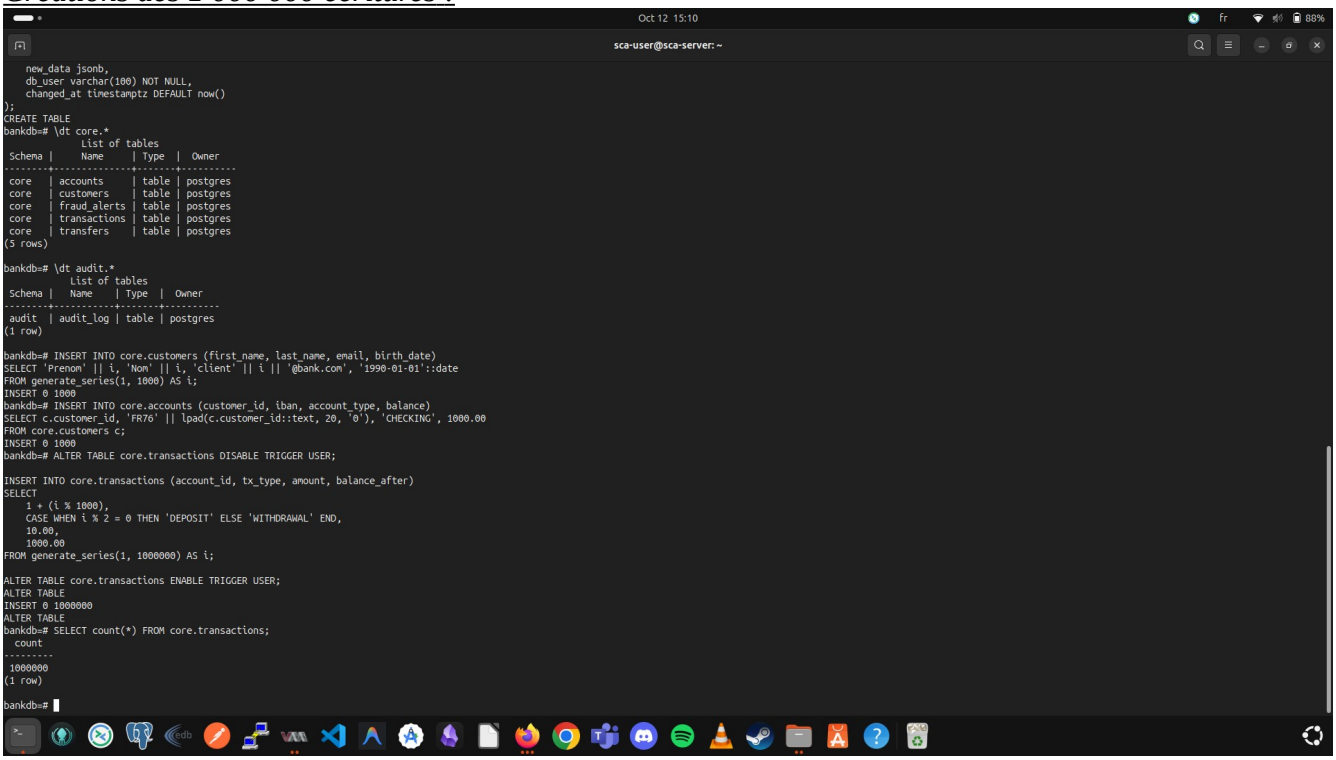
II. Les tables et les données

Création des tables :



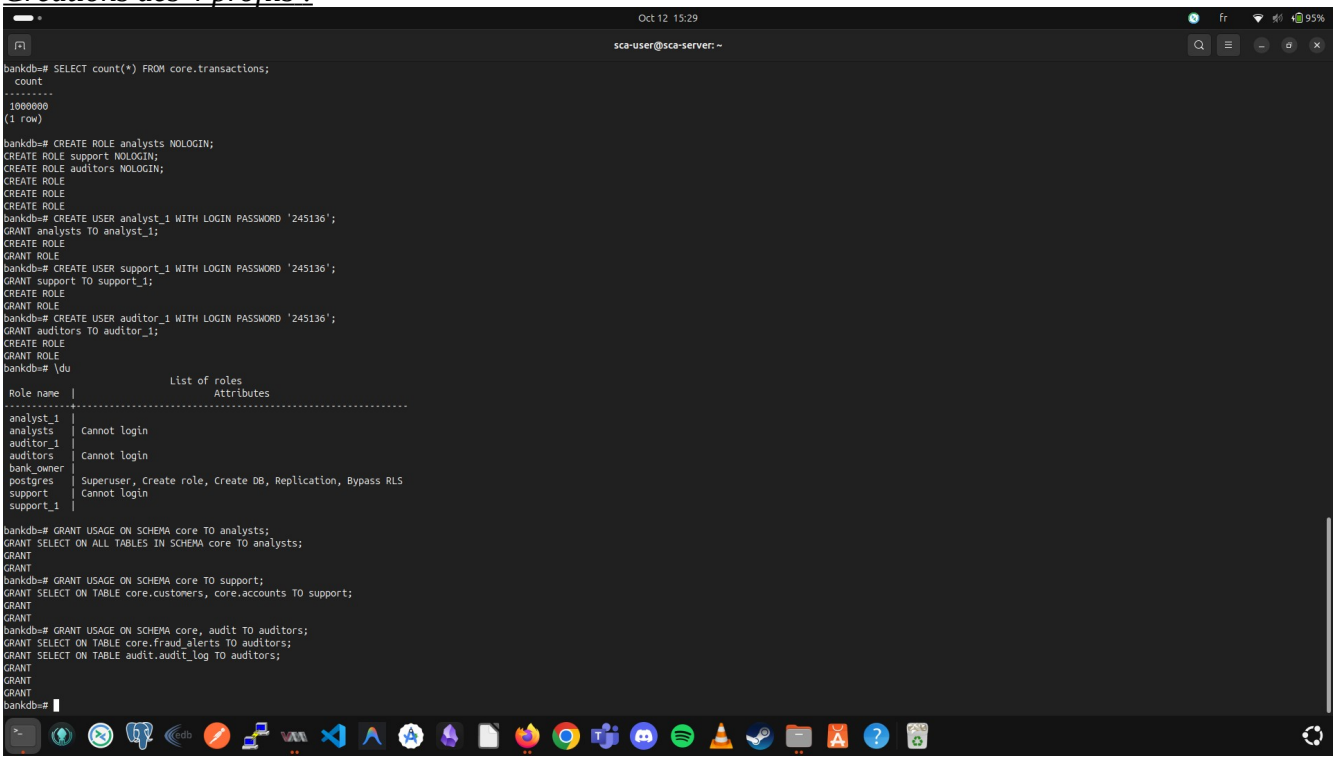


Créations des 1 000 000 écritures :

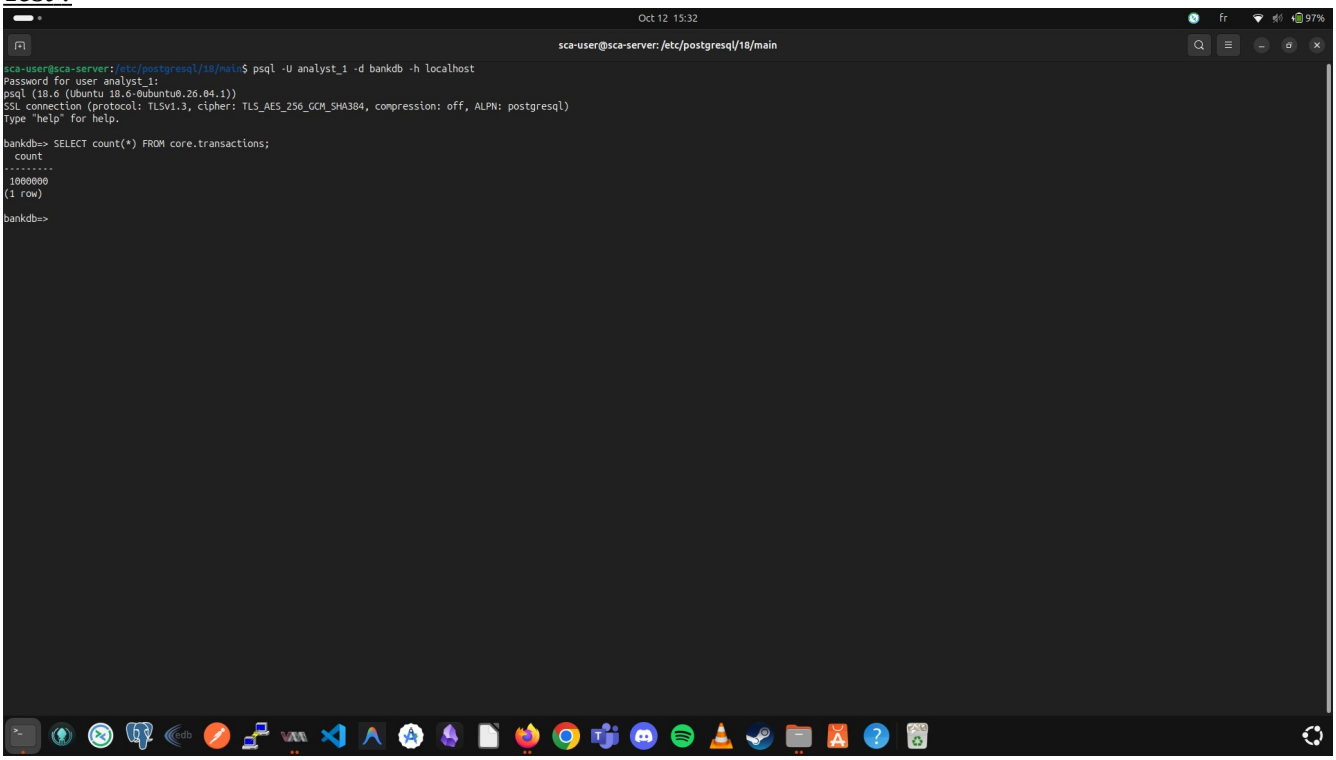


III. Les droits

Créations des 4 profils :

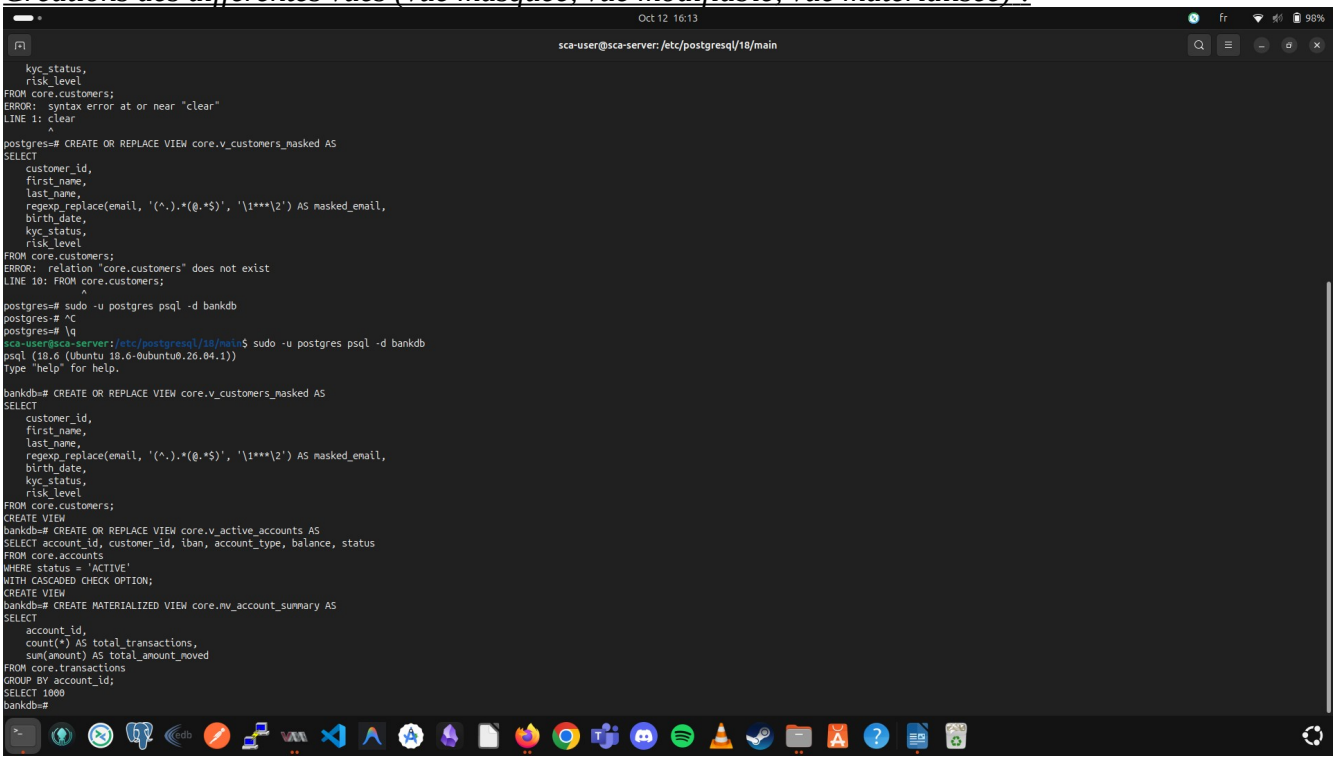


Test :



IV. Les vues

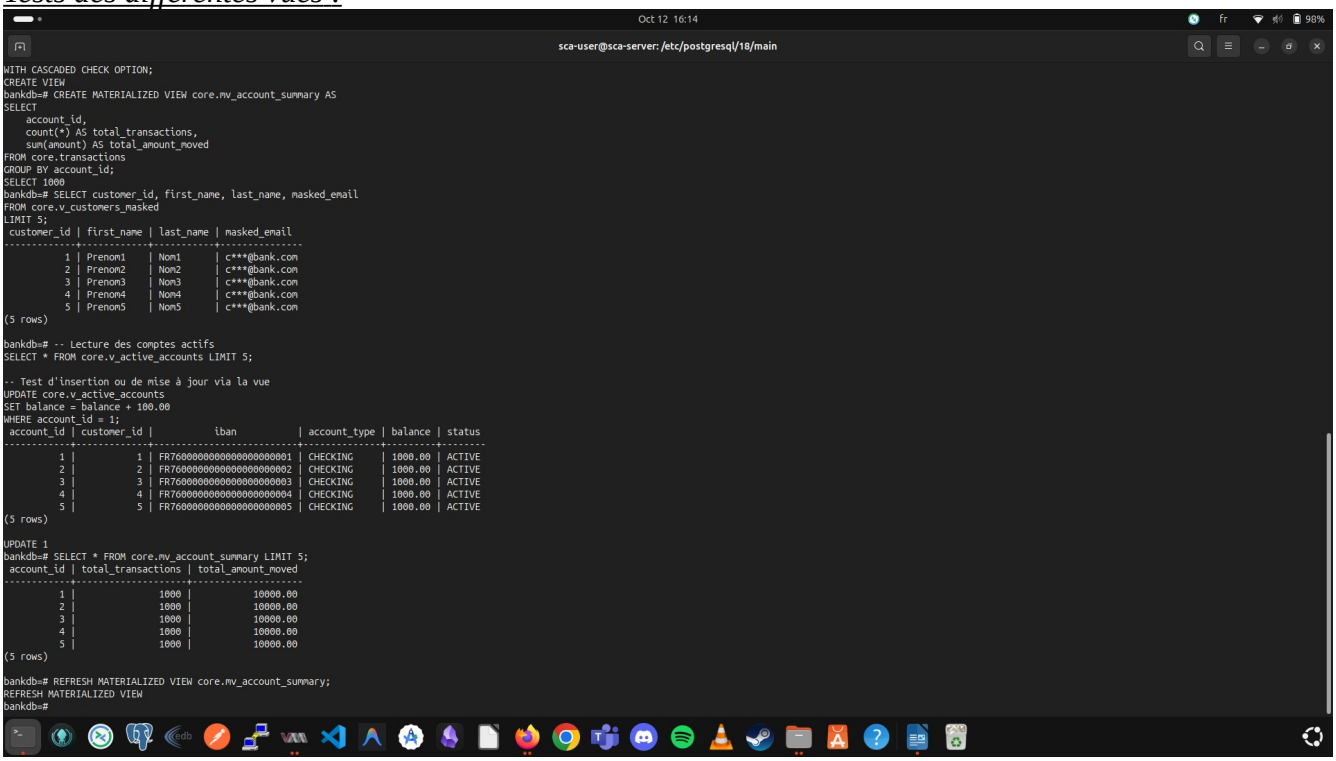
Créations des différentes vues (vue masquée, vue modifiable, vue matérialisée) :



```
Oct 12  16:13                                                    fr  98%
sca-user@sca-server: /etc/postgresql/18/main
    kyc_status,
    risk_level
FROM core.customers;
ERROR:  syntax error at or near "clear"
LINE 1: clear
        ^
postgres=# CREATE OR REPLACE VIEW core.v_customers_masked AS
SELECT
    customer_id,
    first_name,
    last_name,
    regexp_replace(email, '(^.).*(@.*$)', '\1***\2') AS masked_email,
    birth_date,
    kyc_status,
    risk_level
FROM core.customers;
ERROR:  relation "core.customers" does not exist
LINE 10: FROM core.customers;
              ^
postgres=# sudo -u postgres psql -d bankdb
postgres-# ^C
postgres=# \q
sca-user@sca-server:/etc/postgresql/18/main$ sudo -u postgres psql -d bankdb
psql (18.6 (Ubuntu 18.6-0ubuntu0.26.04.1))
Type "help" for help.

bankdb=# CREATE OR REPLACE VIEW core.v_customers_masked AS
SELECT
    customer_id,
    first_name,
    last_name,
    regexp_replace(email, '(^.).*(@.*$)', '\1***\2') AS masked_email,
    birth_date,
    kyc_status,
    risk_level
FROM core.customers;
CREATE VIEW
bankdb=# CREATE OR REPLACE VIEW core.v_active_accounts AS
SELECT account_id, customer_id, iban, account_type, balance, status
FROM core.accounts
WHERE status = 'ACTIVE'
WITH CASCADED CHECK OPTION;
CREATE VIEW
bankdb=# CREATE MATERIALIZED VIEW core.mv_account_summary AS
SELECT
    account_id,
    count(*) AS total_transactions,
    sum(amount) AS total_amount_moved
FROM core.transactions
GROUP BY account_id;
SELECT 1000
bankdb=#
```

Tests des differentes vues :



```
Oct 12  16:14                                                    fr  98%
sca-user@sca-server: /etc/postgresql/18/main
WITH CASCADED CHECK OPTION;
CREATE VIEW
bankdb=# CREATE MATERIALIZED VIEW core.mv_account_summary AS
SELECT
    account_id,
    count(*) AS total_transactions,
    sum(amount) AS total_amount_moved
FROM core.transactions
GROUP BY account_id;
SELECT 1000
bankdb=# SELECT customer_id, first_name, last_name, masked_email
FROM core.v_customers_masked
LIMIT 5;
 customer_id | first_name | last_name | masked_email
-------------+------------+-----------+--------------
           1 | Prenom1    | Nom1      | c***@bank.com
           2 | Prenom2    | Nom2      | c***@bank.com
           3 | Prenom3    | Nom3      | c***@bank.com
           4 | Prenom4    | Nom4      | c***@bank.com
           5 | Prenom5    | Nom5      | c***@bank.com
(5 rows)

bankdb=# -- Lecture des comptes actifs
SELECT * FROM core.v_active_accounts LIMIT 5;

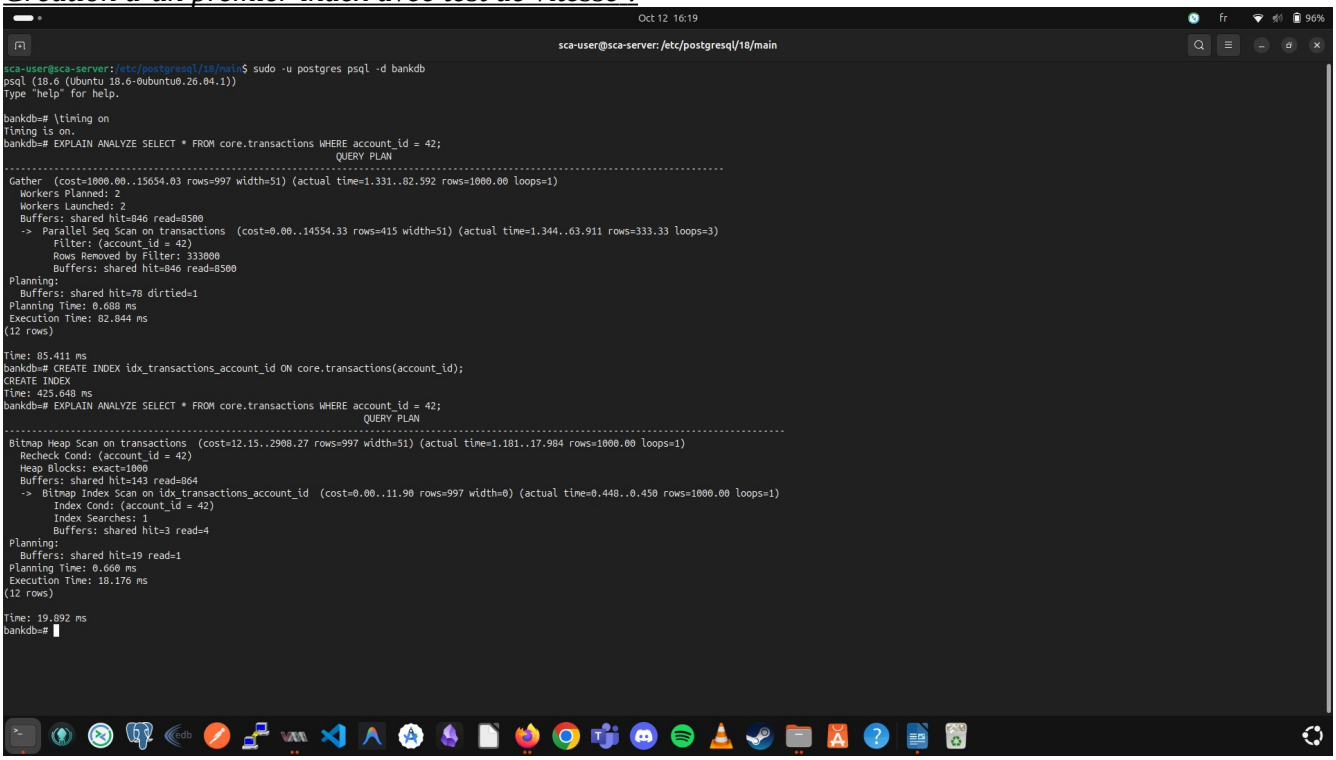
-- Test d'insertion ou de mise à jour via la vue
UPDATE core.v_active_accounts
SET balance = balance + 100.00
WHERE account_id = 1;
 account_id | customer_id |           iban           | account_type | balance | status
------------+-------------+--------------------------+--------------+---------+--------
          1 |           1 | FR7600000000000000000001 | CHECKING     | 1000.00 | ACTIVE
          2 |           2 | FR7600000000000000000002 | CHECKING     | 1000.00 | ACTIVE
          3 |           3 | FR7600000000000000000003 | CHECKING     | 1000.00 | ACTIVE
          4 |           4 | FR7600000000000000000004 | CHECKING     | 1000.00 | ACTIVE
          5 |           5 | FR7600000000000000000005 | CHECKING     | 1000.00 | ACTIVE
(5 rows)

UPDATE 1
bankdb=# SELECT * FROM core.mv_account_summary LIMIT 5;
 account_id | total_transactions | total_amount_moved
------------+--------------------+--------------------
          1 |               1000 |           10000.00
          2 |               1000 |           10000.00
          3 |               1000 |           10000.00
          4 |               1000 |           10000.00
          5 |               1000 |           10000.00
(5 rows)

bankdb=# REFRESH MATERIALIZED VIEW core.mv_account_summary;
REFRESH MATERIALIZED VIEW
bankdb=#
```

V. Les index

Création d'un premier index avec test de vitesse :



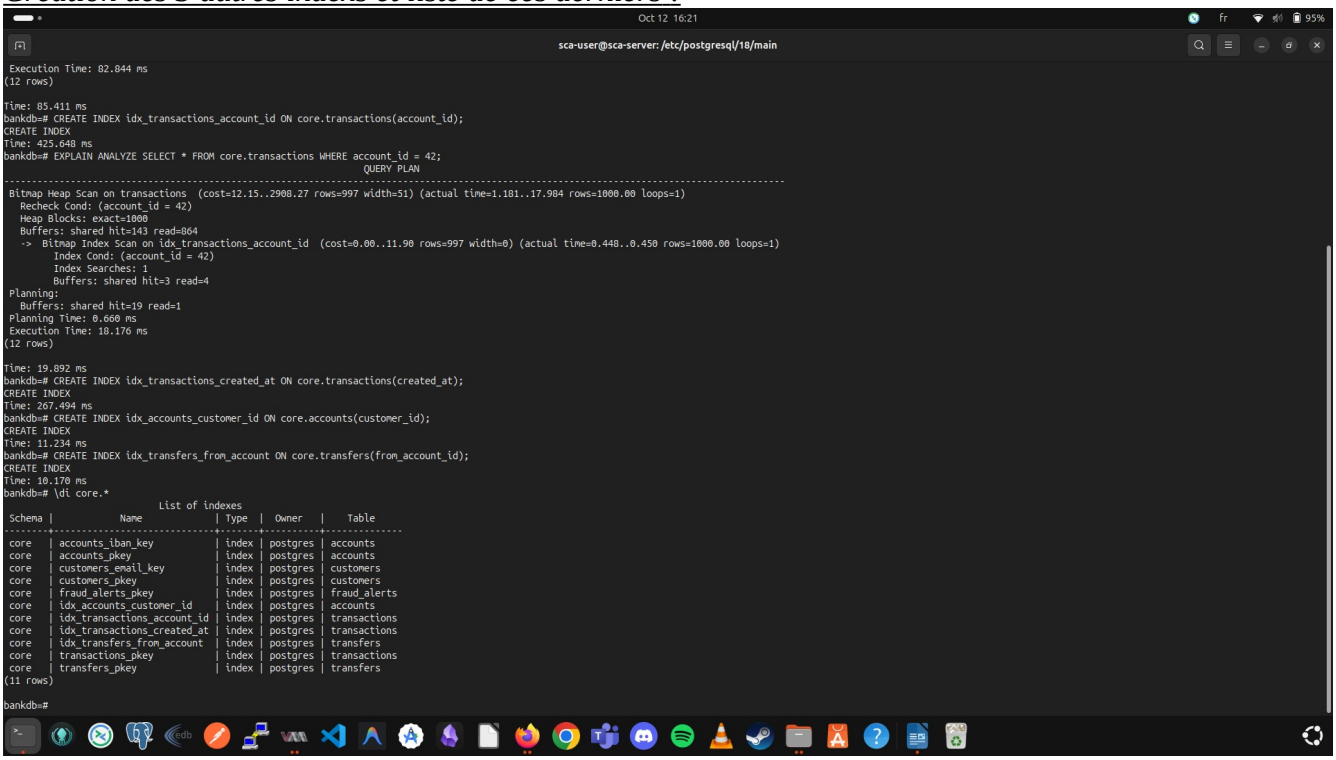

```
sca-user@sca-server:/etc/postgresql/18/main$ sudo -u postgres psql -d bankdb
psql (18.6 (Ubuntu 18.6-0ubuntu0.26.04.1))
Type "help" for help.

bankdb=# \timing on
Timing is on.
bankdb=# EXPLAIN ANALYZE SELECT * FROM core.transactions WHERE account_id = 42;
                                                              QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------
 Gather  (cost=1000.00..15654.03 rows=997 width=51) (actual time=1.331..82.592 rows=1000.00 loops=1)
   Workers Planned: 2
   Workers Launched: 2
   Buffers: shared hit=846 read=8500
   ->  Parallel Seq Scan on transactions  (cost=0.00..14554.33 rows=415 width=51) (actual time=1.344..63.911 rows=333.33 loops=3)
         Filter: (account_id = 42)
         Rows Removed by Filter: 333000
         Buffers: shared hit=846 read=8500
 Planning:
   Buffers: shared hit=78 dirtied=1
 Planning Time: 0.688 ms
 Execution Time: 82.844 ms
(12 rows)

Time: 85.411 ms
bankdb=# CREATE INDEX idx_transactions_account_id ON core.transactions(account_id);
CREATE INDEX
Time: 425.648 ms
bankdb=# EXPLAIN ANALYZE SELECT * FROM core.transactions WHERE account_id = 42;
                                                              QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on transactions  (cost=12.15..2908.27 rows=997 width=51) (actual time=1.181..17.984 rows=1000.00 loops=1)
   Recheck Cond: (account_id = 42)
   Heap Blocks: exact=1000
   Buffers: shared hit=143 read=864
   ->  Bitmap Index Scan on idx_transactions_account_id  (cost=0.00..11.90 rows=997 width=0) (actual time=0.448..0.450 rows=1000.00 loops=1)
         Index Cond: (account_id = 42)
         Index Searches: 1
         Buffers: shared hit=3 read=4
 Planning:
   Buffers: shared hit=19 read=1
 Planning Time: 0.660 ms
 Execution Time: 18.176 ms
(12 rows)

Time: 19.892 ms
bankdb=#
```

Création des 3 autres indexs et liste de ces derniers :




```
 Execution Time: 82.844 ms
(12 rows)

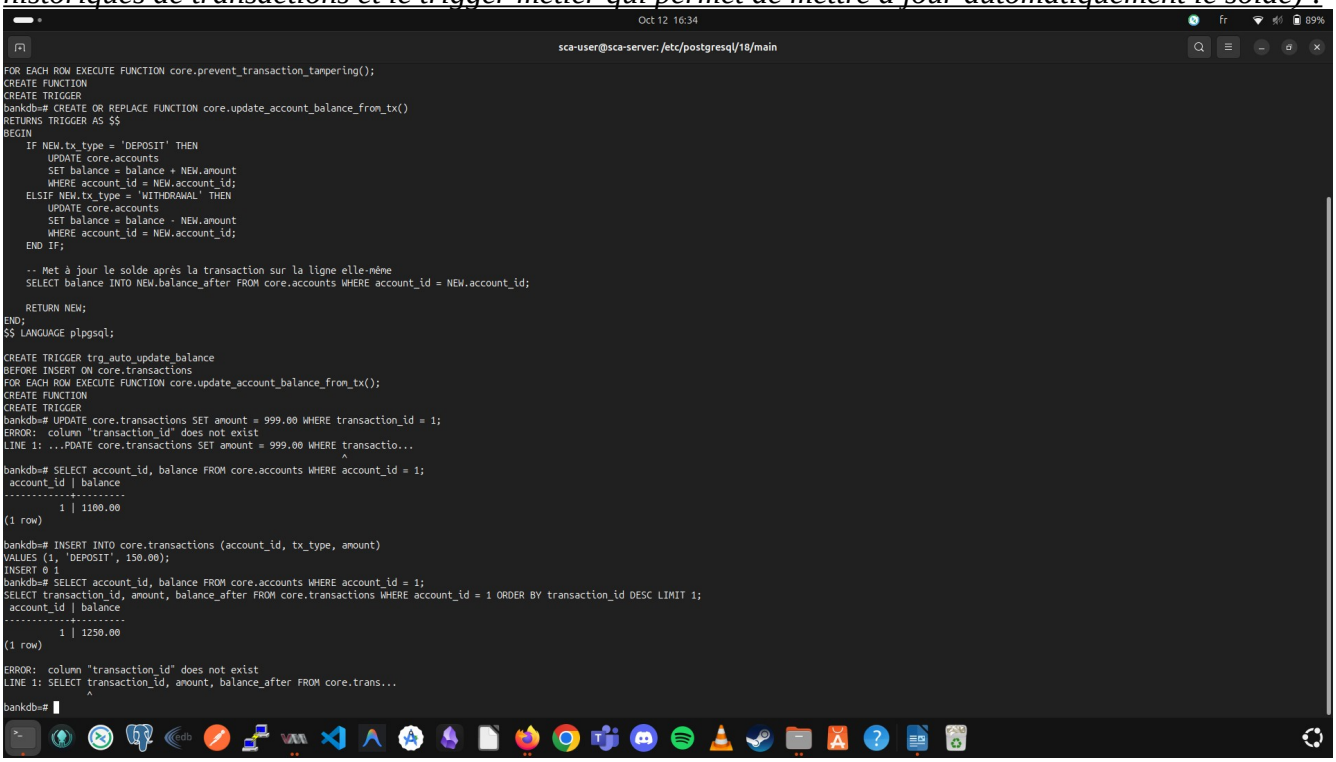
Time: 85.411 ms
bankdb=# CREATE INDEX idx_transactions_account_id ON core.transactions(account_id);
CREATE INDEX
Time: 425.648 ms
bankdb=# EXPLAIN ANALYZE SELECT * FROM core.transactions WHERE account_id = 42;
                                                              QUERY PLAN
---------------------------------------------------------------------------------------------------------------------------------------
 Bitmap Heap Scan on transactions  (cost=12.15..2908.27 rows=997 width=51) (actual time=1.181..17.984 rows=1000.00 loops=1)
   Recheck Cond: (account_id = 42)
   Heap Blocks: exact=1000
   Buffers: shared hit=143 read=864
   ->  Bitmap Index Scan on idx_transactions_account_id  (cost=0.00..11.90 rows=997 width=0) (actual time=0.448..0.450 rows=1000.00 loops=1)
         Index Cond: (account_id = 42)
         Index Searches: 1
         Buffers: shared hit=3 read=4
 Planning:
   Buffers: shared hit=19 read=1
 Planning Time: 0.660 ms
 Execution Time: 18.176 ms
(12 rows)

Time: 19.892 ms
bankdb=# CREATE INDEX idx_transactions_created_at ON core.transactions(created_at);
CREATE INDEX
Time: 267.494 ms
bankdb=# CREATE INDEX idx_accounts_customer_id ON core.accounts(customer_id);
CREATE INDEX
Time: 11.234 ms
bankdb=# CREATE INDEX idx_transfers_from_account ON core.transfers(from_account_id);
CREATE INDEX
Time: 10.170 ms
bankdb=# \di core.*
                        List of indexes
 Schema |            Name             | Type  |  Owner   |    Table
--------+-----------------------------+-------+----------+--------------
 core   | accounts_iban_key           | index | postgres | accounts
 core   | accounts_pkey               | index | postgres | accounts
 core   | customers_email_key         | index | postgres | customers
 core   | customers_pkey              | index | postgres | customers
 core   | fraud_alerts_pkey           | index | postgres | fraud_alerts
 core   | idx_accounts_customer_id    | index | postgres | accounts
 core   | idx_transactions_account_id | index | postgres | transactions
 core   | idx_transactions_created_at | index | postgres | transactions
 core   | idx_transfers_from_account  | index | postgres | transfers
 core   | transactions_pkey           | index | postgres | transactions
 core   | transfers_pkey              | index | postgres | transfers
(11 rows)

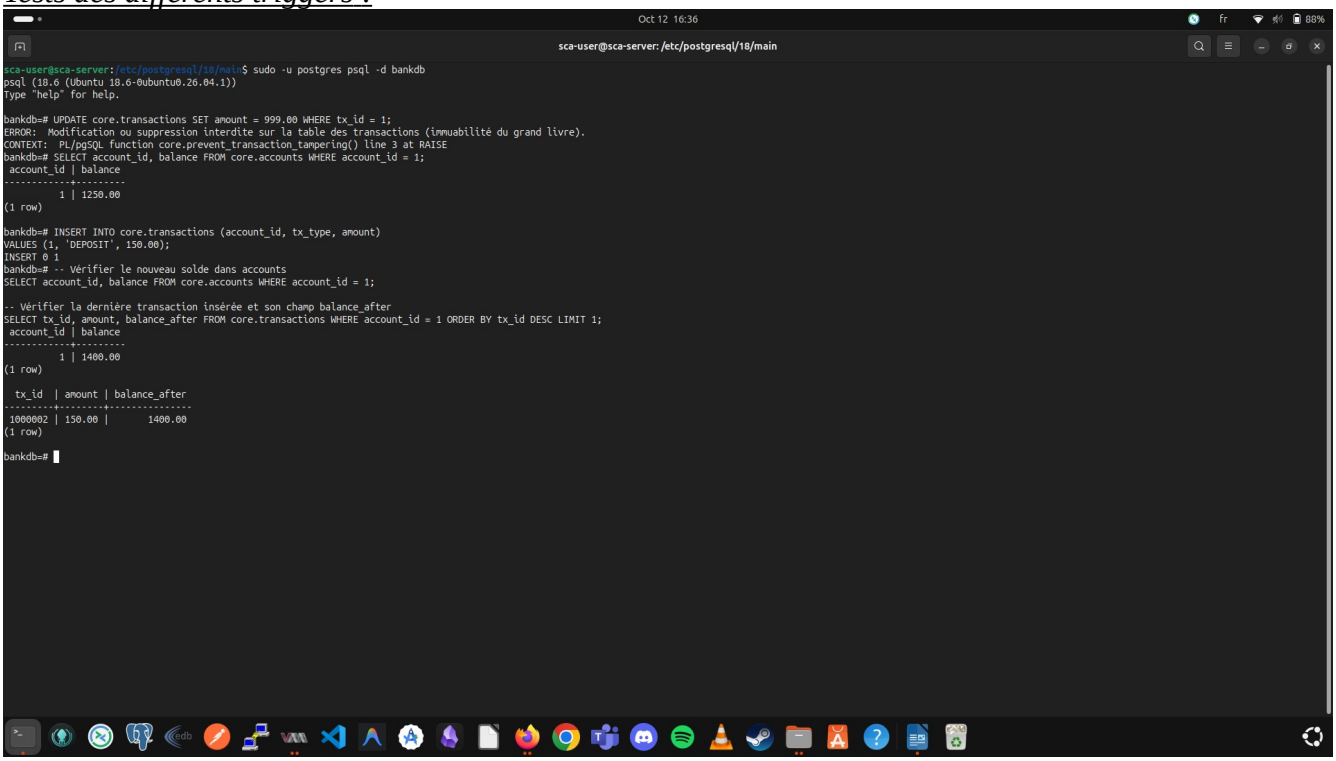
bankdb=#
```

VI. Les triggers

Création des différents triggers (trigger d'immuabilité qui empêche la modification ou suppression des historiques de transactions et le trigger métier qui permet de mettre à jour automatiquement le solde) :

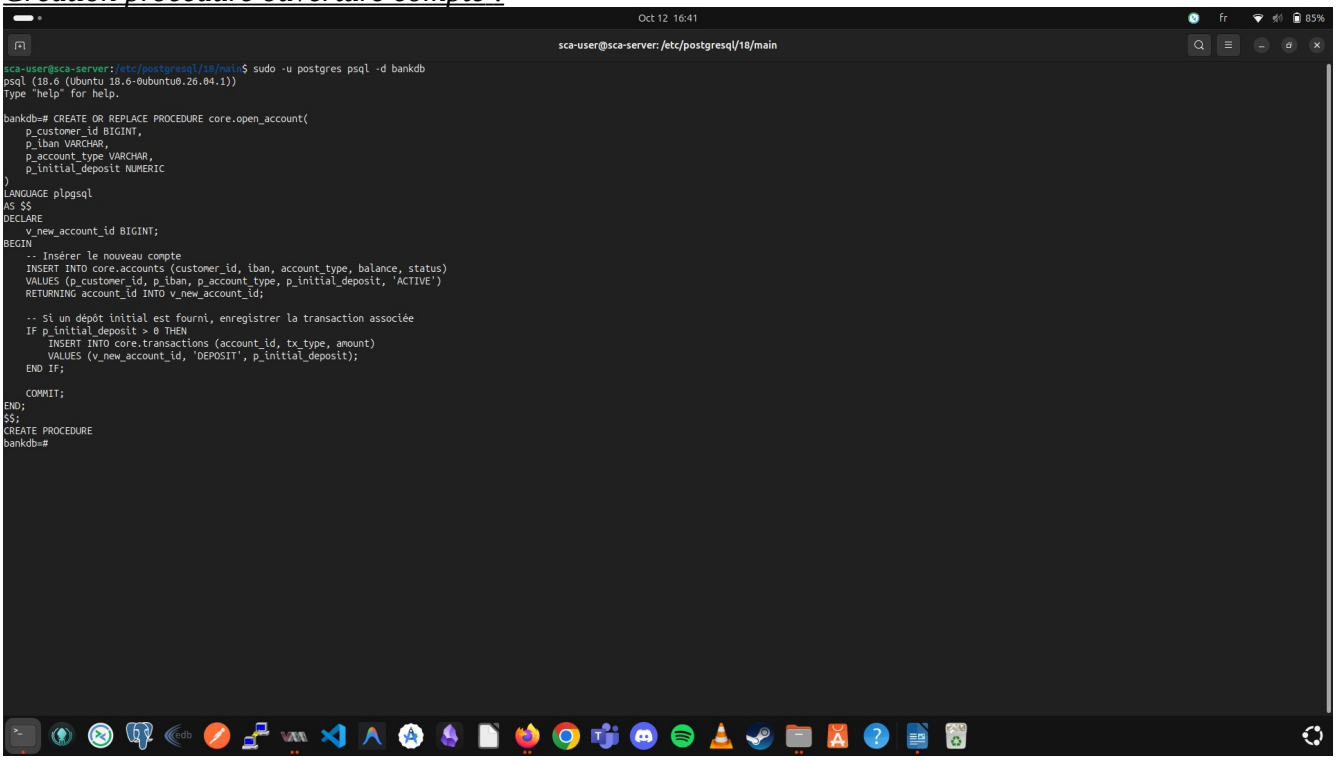


Tests des différents triggers :

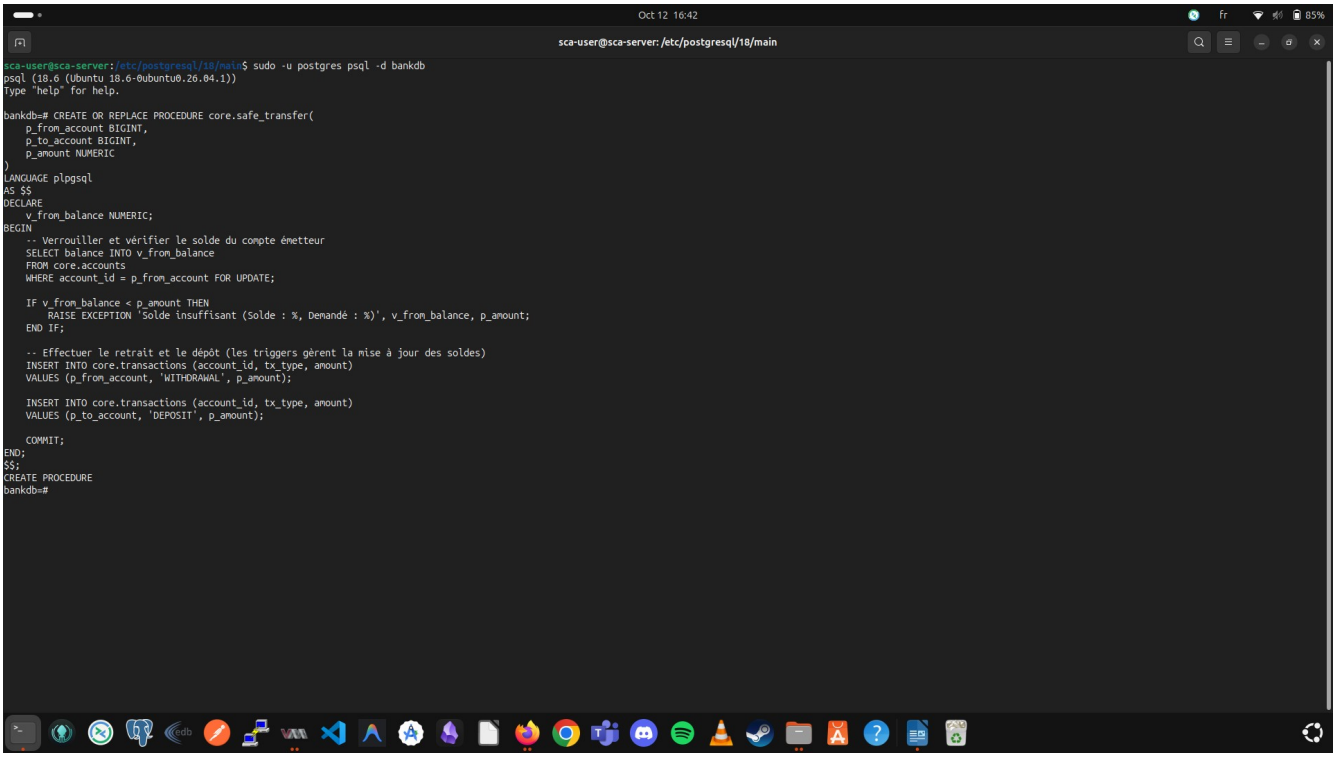


VII. Les procédures

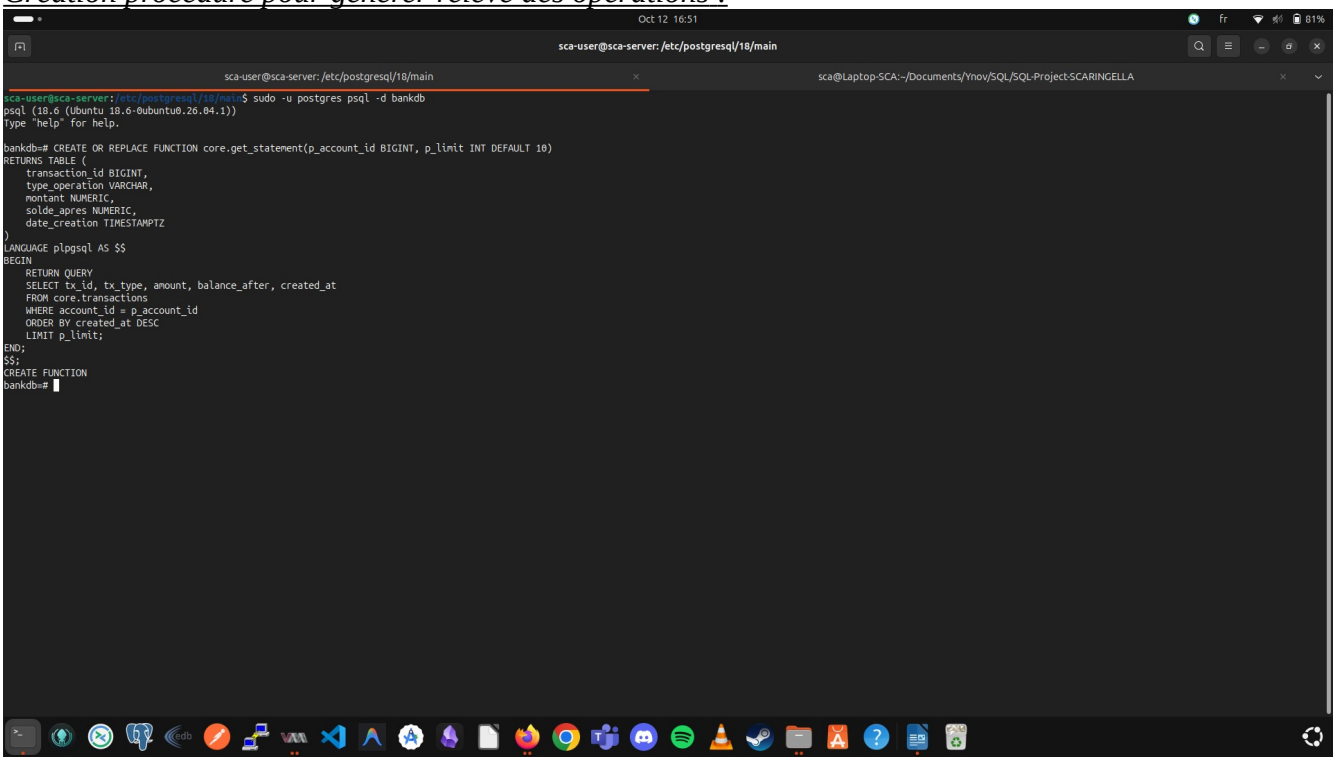
Création procédure ouverture compte :



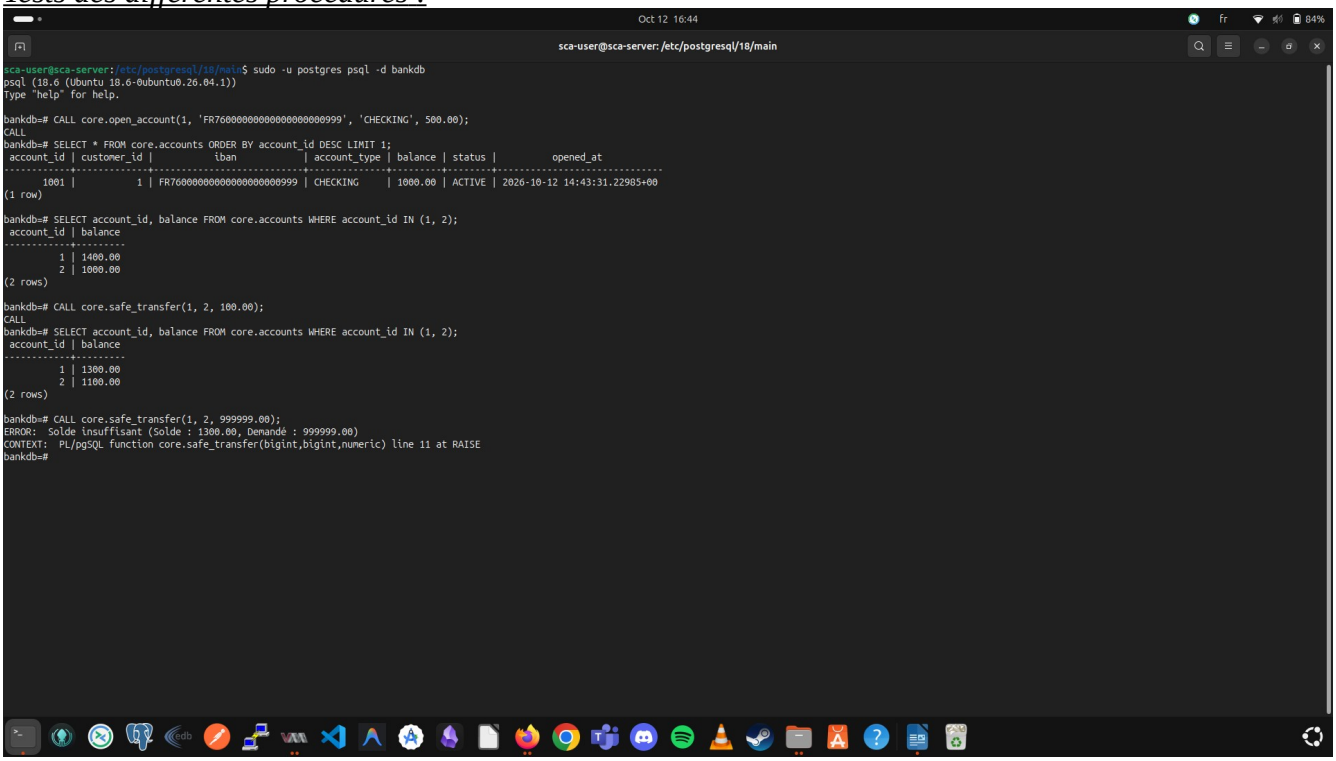
Création procédure de virement sécurisé :

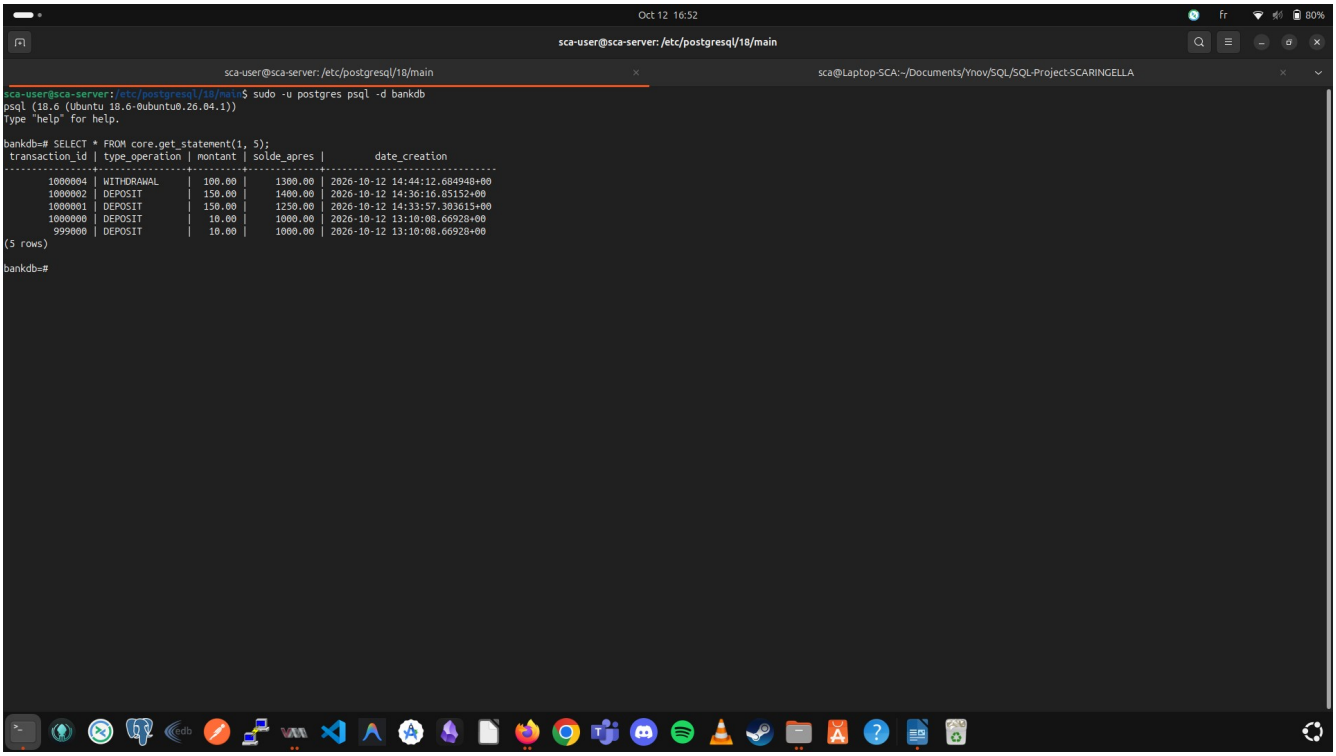


Création procédure pour générer relevé des opérations :



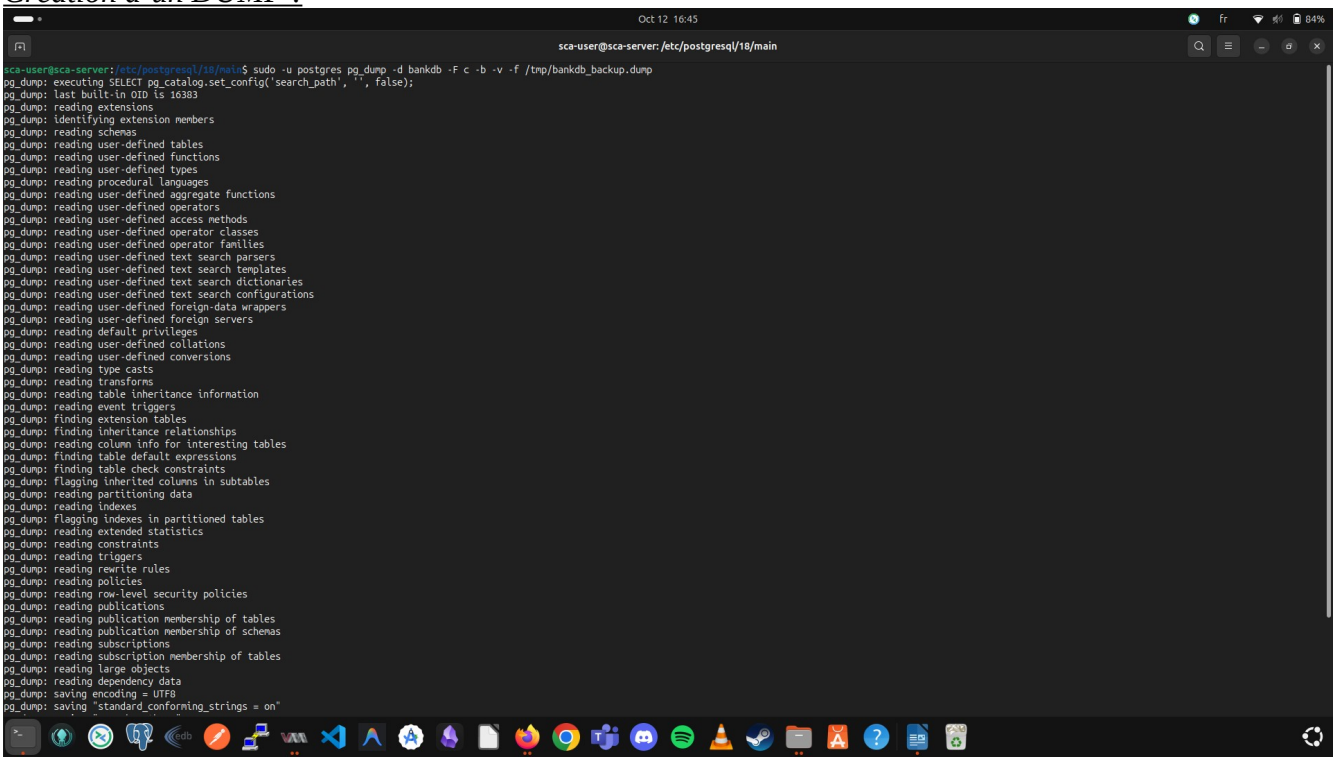
Tests des differentes procédures :



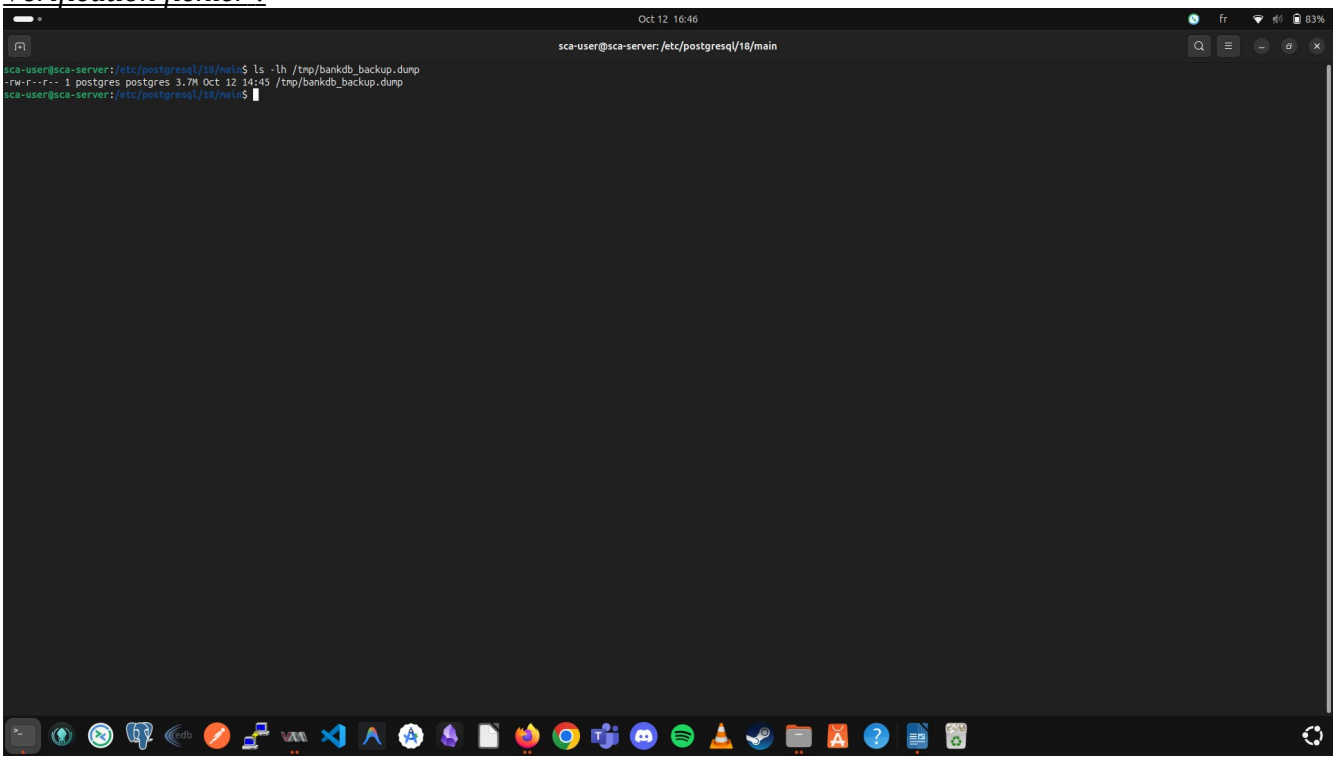

Oct 12 16:52
Fr
80%
sca-user@sca-server: /etc/postgresql/18/main
sca-user@sca-server: /etc/postgresql/18/main
sca@Laptop-SCA: ~/Documents/Ynov/SQL/SQL-Project-SCARINGELLA
sca-user@sca-server:/etc/postgresql/18/main$ sudo -u postgres psql -d bankdb
psql (18.6 (Ubuntu 18.6-0ubuntu0.26.04.1))
Type "help" for help.
bankdb=# SELECT * FROM core.get_statement(1, 5);
transaction_id | type_operation | montant | solde_apres | date_creation
1000004 | WITHDRAWAL | 100.00 | 1300.00 | 2026-10-12 14:44:12.684948+00
1000002 | DEPOSIT | 150.00 | 1400.00 | 2026-10-12 14:36:16.85152+00
1000001 | DEPOSIT | 150.00 | 1250.00 | 2026-10-12 14:33:57.303615+00
1000000 | DEPOSIT | 10.00 | 1000.00 | 2026-10-12 13:10:08.66928+00
999000 | DEPOSIT | 10.00 | 1000.00 | 2026-10-12 13:10:08.66928+00
(5 rows)
bankdb=#

VIII. Tests

Création d'un DUMP :



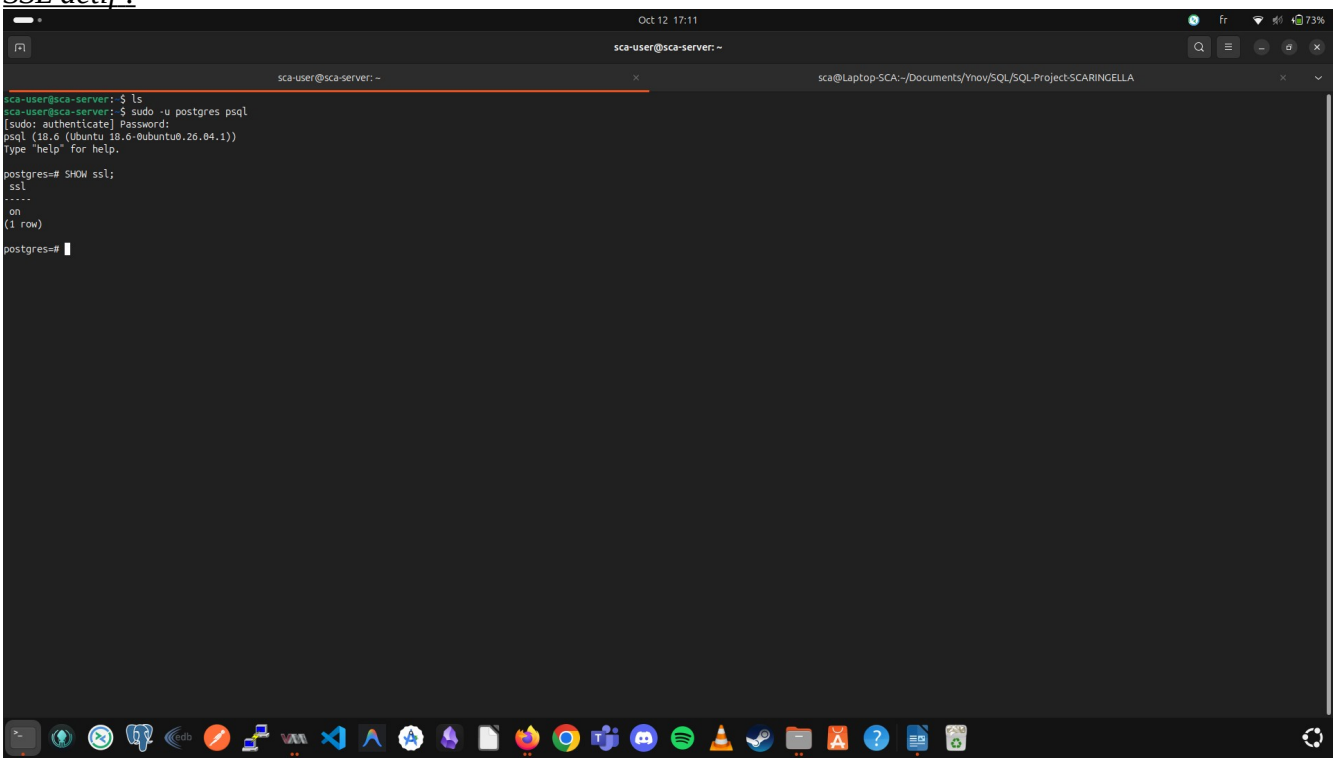
Vérification fichier :



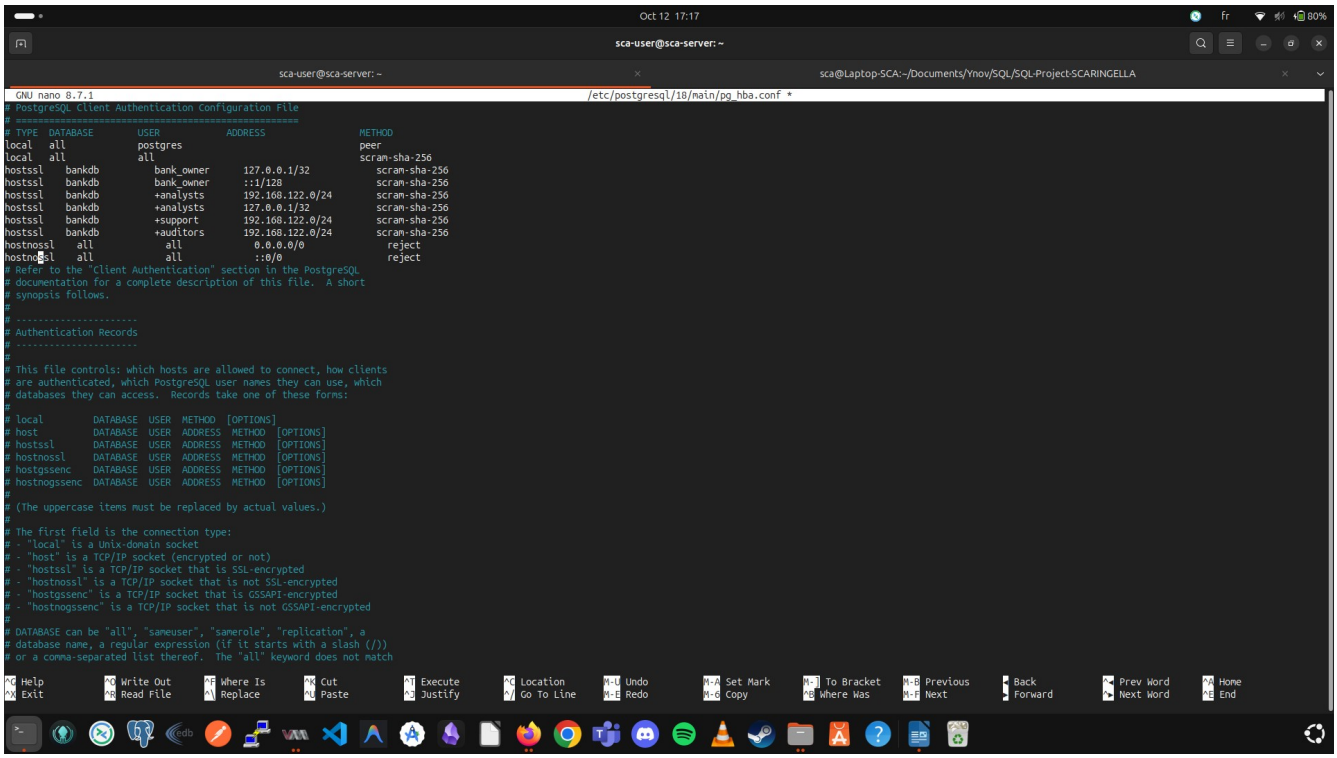
BONUS

I. Chiffrer les connexions à la base (TLS)

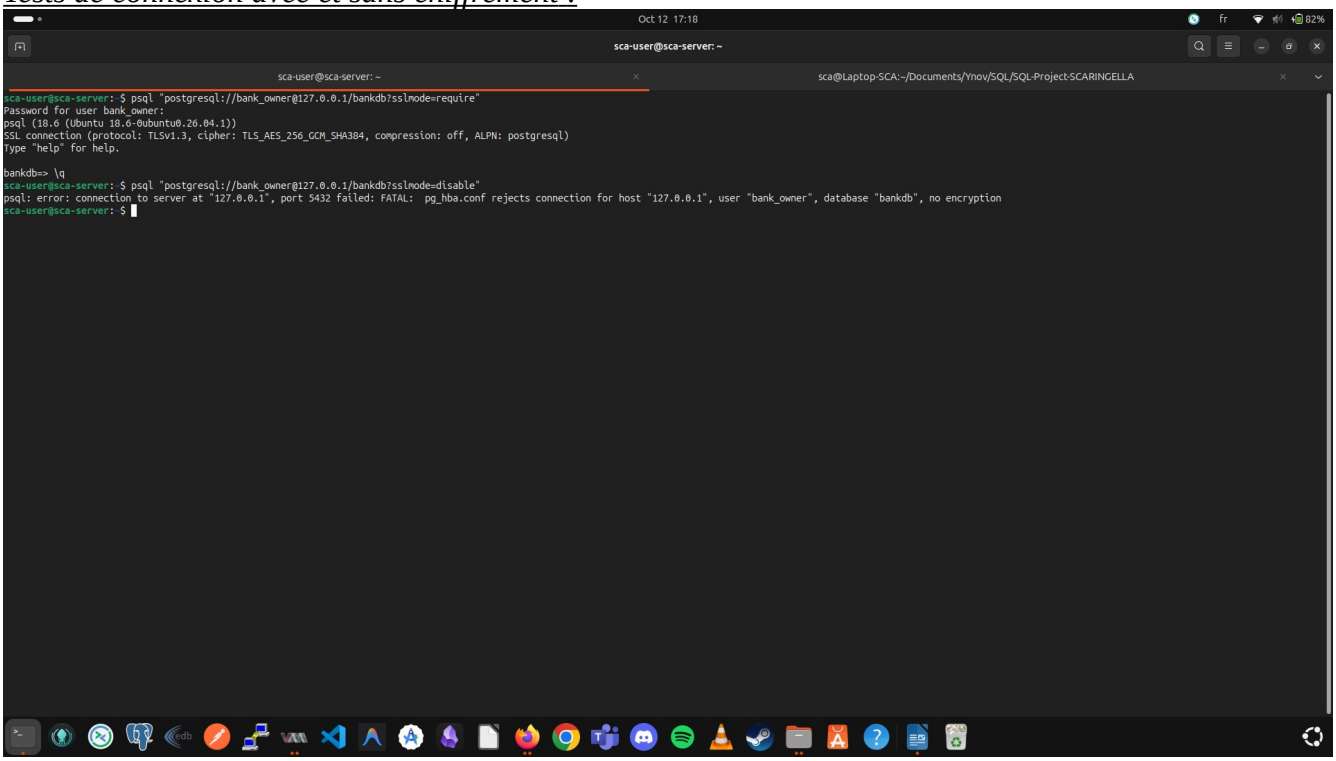
SSL actif :



Modification pg_hba.conf des host en hostssl :

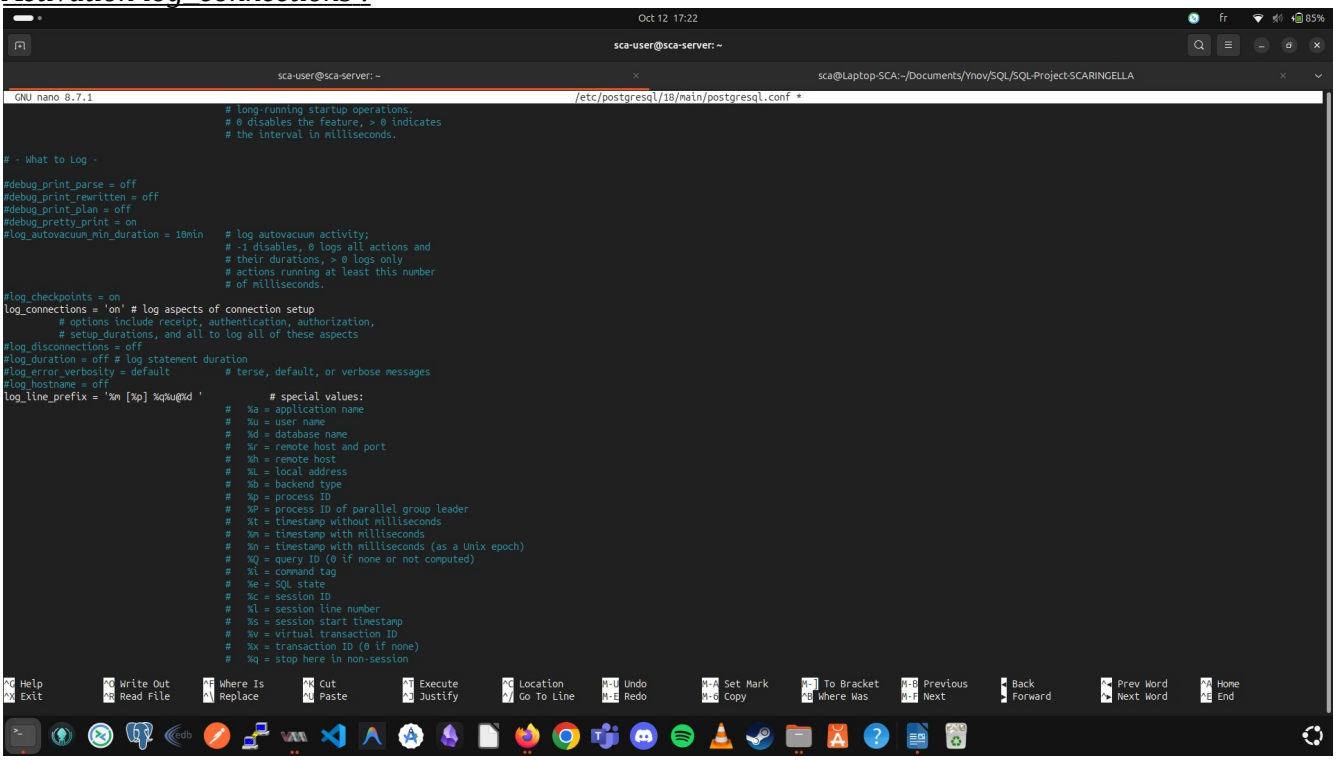


Tests de connexion avec et sans chiffrement :

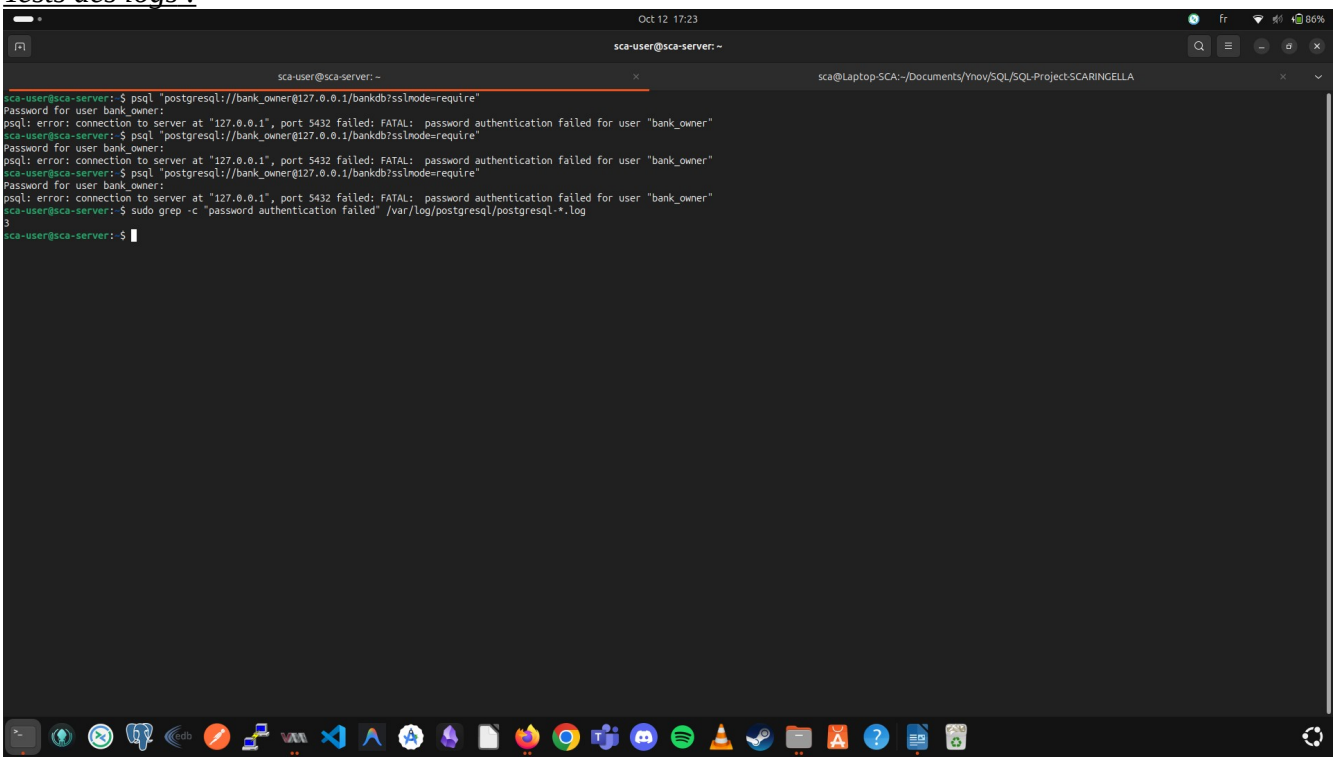


II. Repérer une attaque

Activation log_connections :

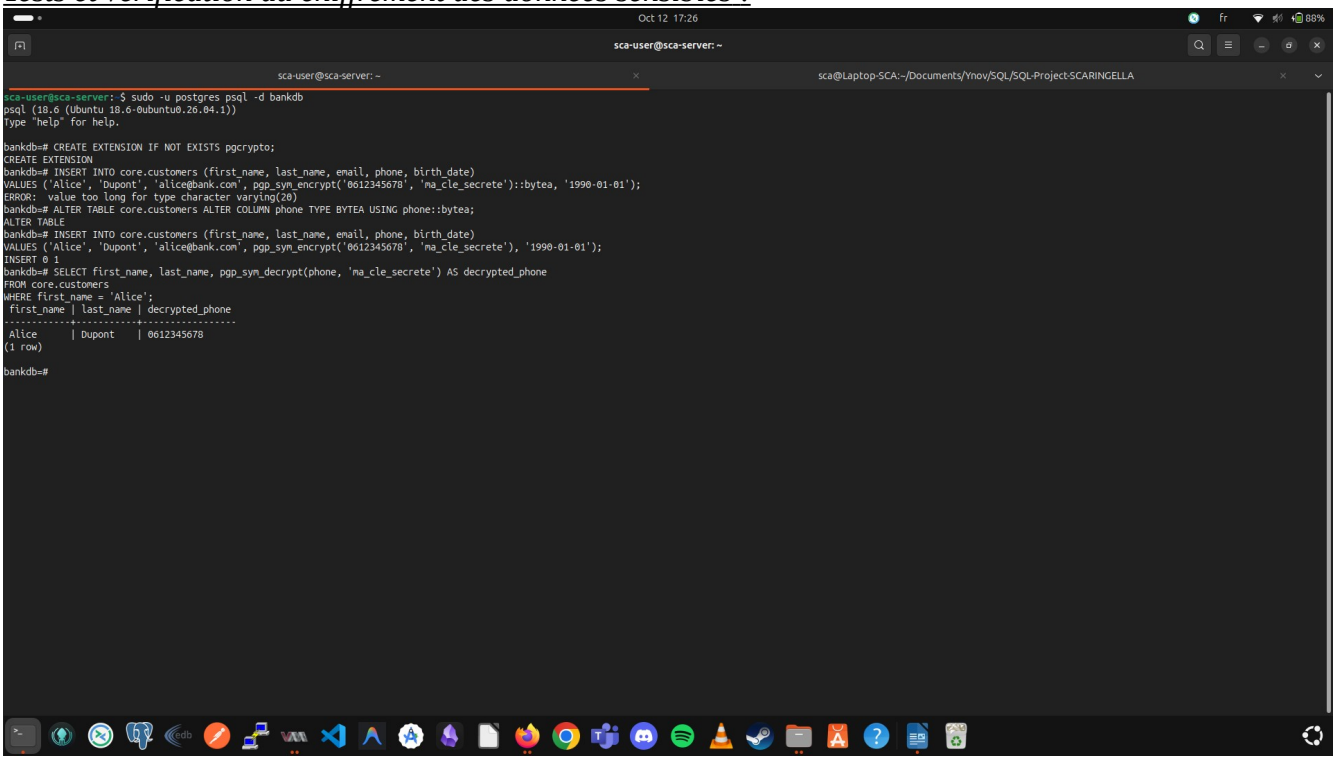


Tests des logs :



III. Cacher une donnée

Tests et vérification du chiffrement des données sensibles :



IV. Attaquer sa base

Démonstration et correction d'une injection SQL :

